Day 74 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 11, Day 5 reference

Watch: The math behind Attention: Keys, Queries, and Values matrices (Luis Serrano Academy) | L19.4.2 Self-Attention and Scaled Dot-Product Attention (Sebastian Raschka)

`day74_self_attention.py`, segment by segment, in the order `main()` uses things. Code boxes are cut from the real script by program (inline comments removed). Day 62's data code and Day 67's training loop are reused verbatim and not repeated. A glossary is at the end.

MATH BOX LEGEND

Symbols: X (m, n, D) = word vectors of m sentences of n positions · Q, K, V (m, n, d) · S, A (m, n, n): scores and weights, row i = position i's query · Z (m, n, d) = outputs

Segment 1 — Self-attention

self_attention

MATH

$$S_{ij} = q_i \cdot k_j; \quad A_i = softmax(S_i) \text{ over real } j; \quad z_i = \Sigma_j A_{ij} v_j$$

CODE

```
def self_attention(X, Wq, Wk, Wv, mask):
    Q, K, Vv = X @ Wq, X @ Wk, X @ Wv
    S = np.einsum("mid,mjd->mij", Q, K)
    A = masked_softmax(S, mask[:, None, :])
    Z = np.einsum("mij,mjd->mid", A, Vv)
    return Z, dict(X=X, Q=Q, K=K, V=Vv, A=A)
```

- `Q, K, Vv = X @ Wq, X @ Wk, X @ Wv` — three projections of the same inputs; @ works on the last axis of a 3-d array.
- `np.einsum("mid,mjd->mij", Q, K)` — for each sentence m, the dot product of query i with key j: an n x n score matrix.
- `mask[:, None, :]` — the key mask, broadcast to every query row: padded keys are masked for everyone.
- `np.einsum("mij,mjd->mid", A, Vv)` — each position's output: its row of weights times the values.

self_attention_backward

MATH

$$dA = dZ\,V^T;\ dV = A^T\,dZ;\ dQ = dS\,K;\ dK = dS^T\,Q;\ dX = dQ\,W_q^T + dK\,W_k^T + dV\,W_v^T$$

CODE

```
def self_attention_backward(dZ, cache, Wq, Wk, Wv):
    X, Q, K, Vv, A = (cache[k] for k in ["X", "Q", "K", "V", "A"])
    dA = np.einsum("mid,mjd->mij", dZ, Vv)
    dV = np.einsum("mij,mid->mjd", A, dZ)
    dS = A * (dA - (A * dA).sum(axis=2, keepdims=True))
    dQ = np.einsum("mij,mjd->mid", dS, K)
    dK = np.einsum("mij,mid->mjd", dS, Q)
    dX = dQ @ Wq.T + dK @ Wk.T + dV @ Wv.T
    flat = lambda T: T.reshape(-1, T.shape[-1])
    return dX, flat(X).T @ flat(dQ), flat(X).T @ flat(dK), flat(X).T @ flat(dV)
```

- `(cache[k] for k in [...])` — a generator unpacked into five names.
- `A * (dA - (A * dA).sum(axis=2, keepdims=True))` — the softmax's backward pass, row by row; zero where a weight is 0 (padding).
- `np.einsum("mij,mid->mjd", dS, Q)` — dK = dS^T Q: each key collects gradient from every query that scored it.
- `flat = lambda T: T.reshape(-1, T.shape[-1])` — every position of every sentence as one row, so one product sums over all of them.

Segment 2 — The tagger

SelfAttnTagger: init and forward

CODE

```
class SelfAttnTagger:
    def __init__(self, vocab_size, n_tags, embed_dim=32, attn_dim=32, hidden_dim=32, lr=1.0,
        clip=1.0, seed=0, attend=True):
        rng = np.random.RandomState(seed)
        D, d, H = embed_dim, attn_dim, hidden_dim
        self.n_tags, self.lr, self.clip, self.attend = n_tags, lr, clip, attend
        self.E = rng.randn(vocab_size, D) * 0.1
        self.Wq = rng.randn(D, d) / np.sqrt(D)
        self.Wk = rng.randn(D, d) / np.sqrt(D)
        self.Wv = rng.randn(D, d) / np.sqrt(D)
        self.W1 = rng.randn(D + d, H) / np.sqrt(D + d)
        self.b1 = np.zeros(H)
        self.Why = rng.randn(H, n_tags) / np.sqrt(H)
        self.by = np.zeros(n_tags)
        self.params = ["E", "Wq", "Wk", "Wv", "W1", "b1", "Why", "by"]
    def forward(self, ids):
        m, T = ids.shape
        self.ids, self.mask = ids, ids != PAD
        self.X = self.E[ids]
        if self.attend:
            self.Z, self.cache = self_attention(self.X, self.Wq, self.Wk, self.Wv, self.mask)
        else:
            self.Z = np.zeros((m, T, self.Wv.shape[1]))
        self.XZ = np.concatenate([self.X, self.Z], axis=2).reshape(m * T, -1)
        self.Hh = np.tanh(self.XZ @ self.W1 + self.b1)
        self.probs = softmax(self.Hh @ self.Why + self.by)
        return self.probs
```

- `attend=True` — a switch: False replaces the attention output with zeros, giving the no-context control.
- `self.mask = ids != PAD` — only padding has id 0, so the mask comes from the ids (Day 68).

- `np.concatenate([self.X, self.Z], axis=2)` — the hidden layer sees the word's own vector AND its context.
- `.reshape(m * T, -1)` — -1 lets NumPy work out the size: here D + d.

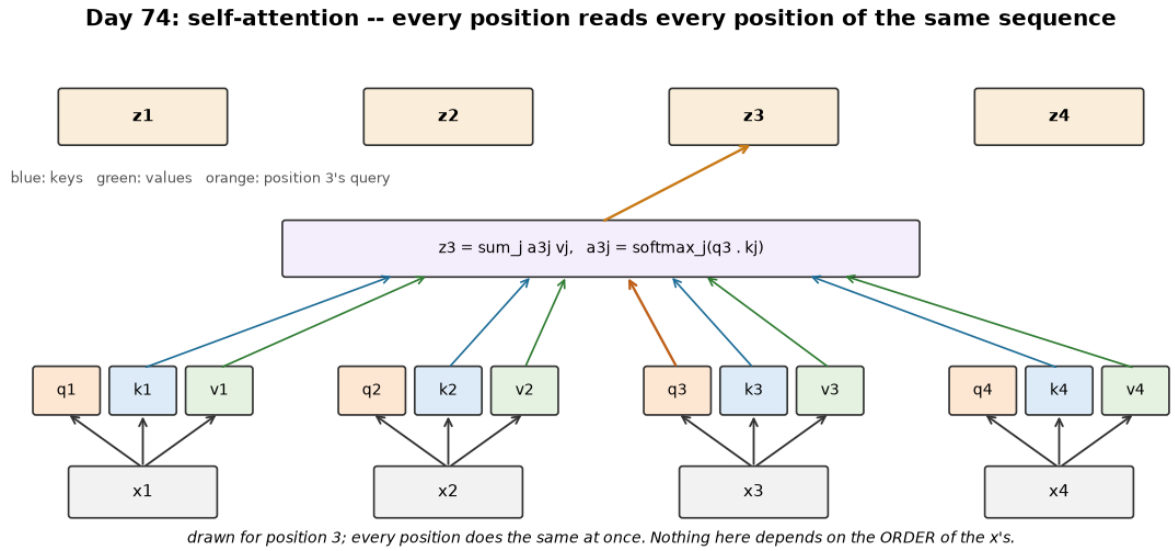
gradients

CODE

```
def gradients(self, tags, mask):
    m, T = self.ids.shape
    D = self.E.shape[1]
    dlog = (self.probs - one_hot(tags.ravel(), self.n_tags)) * mask.ravel()[:, None] / mask.sum()
    g = dict(Why=self.Hh.T @ dlog, by=dlog.sum(axis=0))
    dpre = (dlog @ self.Why.T) * (1 - self.Hh ** 2)
    g["W1"], g["b1"] = self.XZ.T @ dpre, dpre.sum(axis=0)
    dXZ = (dpre @ self.W1.T).reshape(m, T, -1)
    dX = dXZ[:, :, :D].copy()
    if self.attend:
        dXa, g["Wq"], g["Wk"], g["Wv"] = self_attention_backward(dXZ[:, :, D:], self.cache,
            self.Wq, self.Wk, self.Wv)
        dX += dXa
    else:
        g["Wq"], g["Wk"], g["Wv"] = (np.zeros_like(self.Wq), np.zeros_like(self.Wk),
            np.zeros_like(self.Wv))
    g["E"] = np.zeros_like(self.E)
    np.add.at(g["E"], self.ids, dX)
    return g
```

- `dXZ[:, :, :D].copy()` — the part of the gradient for the word vector; .copy() so the next += does not change dXZ.
- `dX += dXa` — the word vectors also made the queries, keys and values, so their gradient has two sources.
- `np.add.at(g["E"], self.ids, dX)` — a word appearing twice gets both gradients added (Day 61).

SYSTEM DIAGRAM



Position 3 reads every position; all positions do the same at once.

Segment 3 — Experiments

The order test

CODE

```
perm = rng.permutation(7)
Zp, _ = self_attention(X[:, perm], Wq, Wk, Wv, mask)
```

- `X[:, perm]` — the same sentences with their positions shuffled.
- `Z[:, perm]` — the original outputs, shuffled the same way: if they match Zp, order does not matter.

The shuffle test and undoing it

CODE

```
p, p_sh = net.predict(ids_te), net.predict(ids_sh)
back = np.empty_like(p_sh)
np.put_along_axis(back, order, p_sh, axis=1)
changed = (back != p)[mask_te].mean()
```

- `np.empty_like(p_sh)` — an array of the same shape, contents not yet set.
- `np.put_along_axis(back, order, p_sh, axis=1)` — write each shuffled prediction back to its word's original position.
- `(back != p)[mask_te].mean()` — the share of real words whose predicted tag changed.

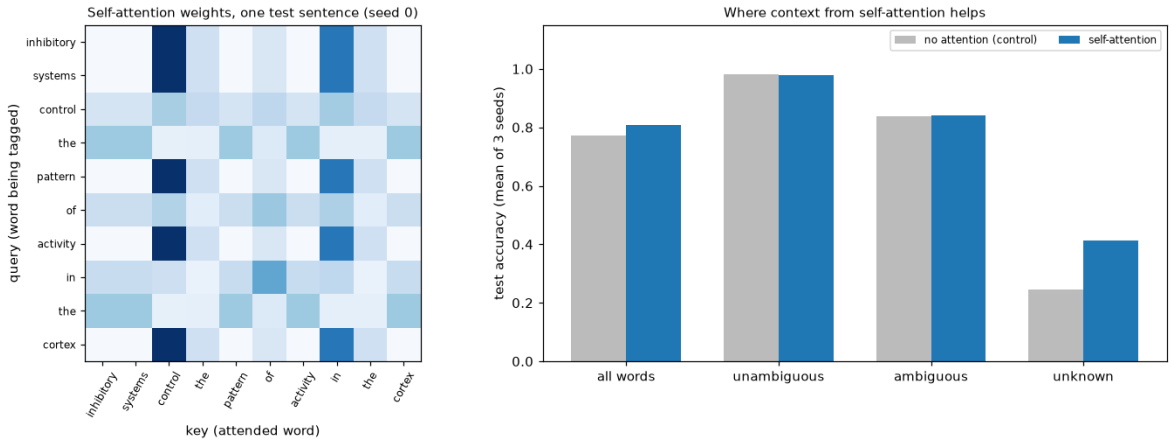
Weight by distance and most attended words

CODE

```
i, j = np.meshgrid(np.arange(A.shape[1]), np.arange(A.shape[2]), indexing="ij")
dist = np.abs(i - j)
valid = mask_te[:, :, None] & mask_te[:, None, :]
uniform = (1.0 / n_real)[:, None, None] * np.ones_like(A)
print(f"\n{'distance':>9}{'mean weight':>13}{'if spread evenly':>18}")
for d in [0, 1, 2, 3, 5, 10]:
    sel = valid & (dist == d)
    print(f"{d:>9}{A[sel].mean():>13.4f}{uniform[sel].mean():>18.4f}")
received = (A * mask_te[:, :, None]).sum(axis=1)
total, count = np.zeros(V), np.zeros(V)
np.add.at(total, ids_te[mask_te], received[mask_te]); np.add.at(count, ids_te[mask_te], 1)
common = np.where(count >= 50)[0]
share = total[common] / count[common]
```

- `np.meshgrid(..., indexing="ij")` — two arrays holding the row index and the column index of every (i, j) pair.
- `mask_te[:, :, None] & mask_te[:, None, :]` — a real query AND a real key.
- `(A * mask_te[:, :, None]).sum(axis=1)` — weight each key receives, summed over the real queries; an even spread gives 1.
- `np.add.at(total, ids_te[mask_te], ...)` — add every occurrence's share into its word's total.

REAL GRAPH



Real graph from the script: one sentence's weights, and accuracy by word group.

Quick Syntax Glossary — new or reinforced constructs

- `np.einsum("mid,mjd->mij", Q, K)` — all pairwise dot products of two sets of vectors, per sentence.
- `x[:, None, :]` — insert an axis so a mask broadcasts over every query.
- `np.take_along_axis / np.put_along_axis` — read from, or write to, positions given by an index array.
- `np.meshgrid(a, b, indexing="ij")` — the row and column index of every cell of a grid.
- `np.empty_like(x)` — an uninitialised array of the same shape and type.
- `x.reshape(-1, d)` — let NumPy compute one dimension.
- `lambda T: ...` — a small function defined inline.

End of Day 74 syntax reference. For the experiments and what they showed, see the Day 74 complete guide.